

[Cloud Computing Concepts Hub](#) › Databases

What is Database Sharding?



What is database sharding?

Database sharding is the process of storing a large database across multiple machines. A single machine or database server, can store and process only a limited amount of data. Database sharding overcomes this limitation by splitting data into smaller chunks, called shards, and storing them across several database servers. All database servers usually have the same underlying technologies, and they work together to store and process large volumes of data.



Why is database sharding important?

As an application grows, the number of application users and the amount of data it stores increase over time. The database becomes a bottleneck if the data volume becomes too large and too many users attempt to use the application to read or save information simultaneously. The application slows down and affects customer experience. Database sharding is one of the methods to solve this problem because it enables parallel processing of smaller datasets across shards.

What are the benefits of database sharding?

Organizations use database sharding to gain the following benefits:

Improve response time



Data retrieval takes longer on a single large database. The [database management system](#) needs to search through many rows to retrieve the correct data. By contrast, data shards have fewer rows than the entire database. Therefore, it takes less time to retrieve specific information, or run a query, from a sharded database.

Avoid total service outage

If the computer hosting the database fails, the application that depends on the database fails too. Database sharding prevents this by distributing parts of the database into different computers. Failure of one of the computers does not shut down the application because it can operate with other functional shards. Sharding is also often done in combination with data replication across shards. So, if one shard becomes unavailable, the data can be accessed and restored from an alternate shard.

Scale efficiently

A growing database consumes more computing resources and eventually reaches storage capacity. Organizations can use database sharding to add more computing resources to support database scaling. They can add new shards at runtime without shutting down the application for maintenance.

How does database sharding work?

A database stores information in multiple datasets consisting of columns and rows. Database sharding splits a single dataset into partitions or shards. Each shard contains unique rows of information that you  can store separately across multiple computers, called nodes. All shards run on separate nodes but share the original database's schema or design.

For example, an unsharded database containing a dataset for customer records might look like this.

Customer ID Name State

1	John	California
2	Jane	Washington
3	Paulo	Arizona
4	Wang	Georgia

Sharding involves separating different rows of information from the table and storing them on different machines, as the following shows.

Computer A

Customer ID Name State

1 John California

2 Jane Washington

Computer B**Customer ID Name State**

3 Paulo Arizona

4 Wang Georgia

Shards

The partitioned data chunks are called logical shards. The machine that stores the logical shard is called a physical shard or database node. A physical shard can contain multiple logical shards.

Shard key

Software developers use a shard key to determine how to partition the dataset. A column in the dataset determines which rows of data group together to form a shard. Database designers choose a shard key from an existing column or create a new one.

Shared-nothing architecture

Database sharding operates on a shared-nothing architecture. Each physical shard operates independently and is unaware of other shards. Only the physical shards that contain the data that you request will process the data in parallel for you.

A software layer coordinates data storage and access from these multiple shards. For example, some types of database technology have automatic sharding features built in. Software developers can also write sharding code in their application to store or retrieve information from the correct shard or shards.

What are the methods of database sharding?

Database sharding methods apply different rules to the shard key to determine the correct node for a particular data row. The following are common sharding architectures.

Range-based sharding

Range-based sharding, or dynamic sharding, splits database rows based on a range of values. Then the database designer assigns a shard key to the respective range. For example, the database designer partitions the data according to the first alphabet in the customer's name as follows.

Name	Shard key
------	-----------

Starts with A to I	A
--------------------	---

Starts with J to S	B
--------------------	---

Starts with T to Z	C
--------------------	---

When it is writing a customer record to the database, the application determines the correct shard key by checking the customer's name. Then the application matches the key to its physical node and stores the row on that machine. Similarly, the application performs a reverse match when searching for a particular record.

Pros and cons



Depending on the data values, range-based sharding can result in the overloading of data on a single physical node. In our example, shard A (containing names that start with A to I) might contain a much larger number of rows of data than shard C (containing names that start with T to Z). However, it is easier to implement.

Hashed sharding

Hashed sharding assigns the shard key to each row of the database by using a mathematical formula called a hash function. The hash function takes the information from the row and produces a hash value. The application uses the hash value as a shard key and stores the information in the corresponding physical shard.

Software developers use hashed sharding to evenly distribute information in a database among multiple shards. For example, the software separates customer records into two shards with alternative hash values of 1 and 2.

Name	Hash value
------	------------

John	1
------	---

Jane 2

Paulo 1

Wang 2

Pros and cons

Although hashed sharding results in even data distribution among physical shards, it does not separate the database based on the meaning of the information. Therefore, software developers might face difficulties reassigning the hash value when adding more physical shards to the computing environment.

Directory sharding

Directory sharding uses a lookup table to match database information to the corresponding physical shard. A lookup table is like a table on a spreadsheet that links a database column to a shard key. For example, the following diagram shows a lookup table for clothing colors.

Color	Shard key
Blue	A
Red	B
Yellow	C
Black	D



When an application stores clothing information in the database, it refers to the lookup table. If a dress is blue, the application stores the information in the corresponding shard.

Pros and cons

Software developers use directory sharding because it is flexible. Each shard is a meaningful representation of the database and is not limited by ranges. However, directory sharding fails if the lookup table contains the wrong information.

Geo sharding

Geo sharding splits and stores database information according to geographical location. For example, a dating service website uses a database to store customer information from various cities as follows.

Name Shard key

John California

Jane Washington

Paulo Arizona

Software developers use cities as shard keys. They store each customer's information in physical shards that are geographically located in the respective cities.

Pros and cons

Geo sharding allows applications to retrieve information faster due to the shorter distance between the shard and the customer making the request. If data access patterns are predominantly based on geography, then this works well. However, geo sharding can also result in uneven data distribution.

How to optimize database sharding for even data distribution



When a data overload occurs on specific physical shards although others remain underloaded, it results in database hotspots. Hotspots slow down the retrieval process on the database, defeating the purpose of data sharding.

Good shard-key selection can evenly distribute data across multiple shards. When choosing a shard key, database designers should consider the following factors.

Cardinality

Cardinality describes the possible values of the shard key. It determines the maximum number of possible shards on separate column-oriented databases. For example, if the database designer chooses a yes/no data field as a shard key, the number of shards is restricted to two.

Frequency

Frequency is the probability of storing specific information in a particular shard. For example, a database designer chooses age as a shard key for a fitness website. Most of the records might go into nodes for subscribers aged 30–45 and result in database hotspots.

Monotonic change

Monotonic change is the rate of change of the shard key. A monotonically increasing or decreasing shard key results in unbalanced shards. For example, a feedback database is split into three different physical

shards as follows:

- Shard A stores feedback from customers who have made 0–10 purchases.
- Shard B stores feedback from customers who have made 11–20 purchases.
- Shard C stores feedback from customers who have made 21 or more purchases.

As the business grows, customers will make more than 21 or more purchases. The application stores their feedback in Shard C. This results in an unbalanced shard because Shard C contains more feedback records than other shards.

What are the alternatives to database sharding?

Database sharding is a horizontal scaling strategy that allocates additional nodes or computers to share the workload of an application. Organizations benefit from horizontal scaling because of its fault-tolerant architecture. When one computer fails, the others continue to operate without disruption. Database designers reduce downtime by spreading logical shards across multiple servers.

However, sharding is one among several other database scaling strategies. Explore some other techniques and understand how they compare.

Vertical scaling

Vertical scaling increases the computing power of a single machine. For example, the IT team adds a CPU,  RAM, and a hard disk to a database server to handle increasing traffic.

Comparison of database sharding and vertical scaling

Vertical scaling is less costly, but there is a limit to the computing resources you can scale vertically. Meanwhile, sharding, a horizontal scaling strategy, is easier to implement. For example, the IT team installs multiple computers instead of upgrading old computer hardware.

Replication

Replication is a technique that makes exact copies of the database and stores them across different computers. Database designers use replication to design a fault-tolerant [relational database management system](#). When one of the computers hosting the database fails, other replicas remain operational. Replication is a common practice in distributed computing systems.

Comparison of database sharding and replication

Database sharding does not create copies of the same information. Instead, it splits one database into multiple parts and stores them on different computers. Unlike replication, database sharding does not result in [high availability](#). Sharding can be used in combination with replication to achieve both scale and high availability.

In some cases, database sharding might consist of replications of specific datasets. For example, a retail store that sells products to both US and European customers might store replicas of size conversion tables on different shards for both regions. The application can use the duplicate copies of the conversion table to convert the measurement size without accessing other database servers.

Partitioning

Partitioning is the process of splitting a database table into multiple groups. Partitioning is classified into two types:

- Horizontal partitioning splits the database by rows.
- Vertical partitioning creates different partitions of the database columns.

Comparison of database sharding and partitioning

Database sharding is like horizontal partitioning. Both processes split the database into multiple groups of unique rows. Partitioning stores all data groups in the same computer, but database sharding spreads them across different computers.

What are the challenges of database sharding?

Organizations might face these challenges when implementing database sharding.



Data hotspots

Some of the shards become unbalanced due to the uneven distribution of data. For example, a single physical shard that contains customer names starting with A receives more data than others. This physical shard will use more computing resources than others.

Solution

You can distribute data evenly by using optimal shard keys. Some datasets are better suited for sharding than others.

Operational complexity

Database sharding creates operational complexity. Instead of managing a single database, developers have to manage multiple database nodes. When they are retrieving information, developers must query several shards and combine the pieces of information together. These retrieval operations can complicate analytics.

Solution

In the [AWS database portfolio](https://aws.amazon.com/what-is/database-sharding/), database setup and operations have been automated to a large extent. This makes working with a sharded database architecture a more streamlined task.

Infrastructure costs

Organizations pay more for infrastructure costs when they add more computers as physical shards. Maintenance costs can add up if you increase the number of machines in your on-premises data center.

Solution

Developers use [Amazon Elastic Compute Cloud \(Amazon EC2\)](#) to host and scale shards in the cloud. You can save money by using virtual infrastructure that AWS fully manages.

Application complexity

Most database management systems do not have built-in sharding features. This means that database designers and software developers must manually split, distribute, and manage the database.

Solution

You can [migrate your data](#) to the appropriate [AWS purpose-built databases](#), which have several built-in features that support horizontal scaling.

How can AWS help with database sharding?

AWS is a global [data management platform](#) that you can use to build a modern data strategy. With AWS, you can choose the right [purpose-built database](#), achieve performance at scale, run fully managed databases, and rely on high availability and security.



Get started with data management on AWS by [creating an AWS account](#) today.

AWS next steps

Check out additional product-related resources

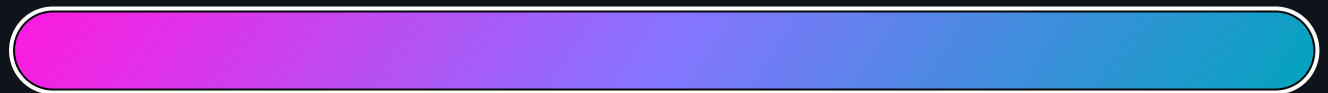




Learn

Resources

Developers

Help

Amazon is an Equal Opportunity Employer: Minority / Women / Disability / Veteran / Gender Identity / Sexual Orientation / Age.



Back to top 

[Privacy](#) [Site terms](#) [Cookie Preferences](#)

© 2025, Amazon Web Services, Inc. or its affiliates. All rights reserved.